



Fate: Fast Edge Inference of Mixture-of-Experts Models via Cross-Layer Gate

Zhiyuan Fang
Zhuhai Key Laboratory of Trusted
Large Language Models, Sun Yat-sen
University
Zhuhai, China
fangzhy27@mail2.sysu.edu.cn

Xingfan Yu
Sun Yat-sen University
Zhuhai, China
yuxf8@mail2.sysu.edu.cn

Yuegui Huang
Sun Yat-sen University
Guangzhou, China
huangyg35@mail3.sysu.edu.cn

Zicong Hong*
Hong Kong University of Science and
Technology
Hong Kong, China
ziconghong@gmail.com

Yufeng Lyu
Huawei Technologies Co. Ltd
Beijing, China
lvufeng1@huawei.com

Wuhui Chen
Sun Yat-sen University
Zhuhai, China
Peng Cheng Laboratory
Shenzhen, China
chenwuh@mail.sysu.edu.cn

Yue Yu
Peng Cheng Laboratory
Shenzhen, China
yuy@pcl.ac.cn

Fan Yu
Huawei Technologies Co. Ltd
Hangzhou, China
fan.yu@huawei.com

Abstract

With the rapid growth and rising complexity of web content, edge-deployed LLMs have become essential for enhancing users' online experiences. However, sparsely-activated Mixture-of-Experts (MoE) models, which are well-suited for edge scenarios, face significant memory bottleneck challenges. Offload-based methods have been proposed to mitigate the problem, but they face difficulties with expert prediction. To promote the application of MoE models in edge scenarios, we propose Fate, an offloading system designed for MoE models to enable efficient inference in resource-constrained environments. The key insight behind Fate is that gate inputs from adjacent layers can be effectively used for expert prefetching, achieving high prediction accuracy. Furthermore, Fate employs a shallow-favoring expert caching strategy that increases the expert hit rate to 99%. Additionally, Fate integrates tailored quantization strategies for cache optimization and I/O efficiency. Experimental results show that, compared to baselines, Fate achieves up to $1.34\times$ – $5.07\times$ prefill speedup and $1.26\times$ – $4.41\times$ decoding speedup, while maintaining inference quality.

CCS Concepts

• **Computing methodologies** → **Natural language generation.**

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792527>

Keywords

Mixture-of-Experts; Offloading; LLM Inference

ACM Reference Format:

Zhiyuan Fang, Xingfan Yu, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, and Fan Yu. 2026. Fate: Fast Edge Inference of Mixture-of-Experts Models via Cross-Layer Gate. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792527>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18332847>.

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across diverse web applications [5, 32, 40]. The most advanced models [1, 3, 36] are typically deployed on large-scale clusters equipped with cutting-edge GPUs. However, as web content continues to proliferate, on-device LLMs have emerged as an effective solution for supporting users' online activities, offering additional advantages such as stronger privacy protection, enhanced personalization, and reduced inference latency. They have been successfully applied in domains such as web browsing [6, 22], online shopping [27], and AI smartphones [20, 28, 43], significantly improving the overall user experience.

Current research and applications of edge LLMs focus primarily on dense models [13, 39, 42]. Benefiting from the accelerated growth in the knowledge density of language models, 3-billion-parameter models are well suited for providing high-quality inference services [42]. Meanwhile, Mixture of Experts (MoE) models [15, 29], are actually well suited for edge scenarios because of their sparse activation characteristics. They can achieve superior performance

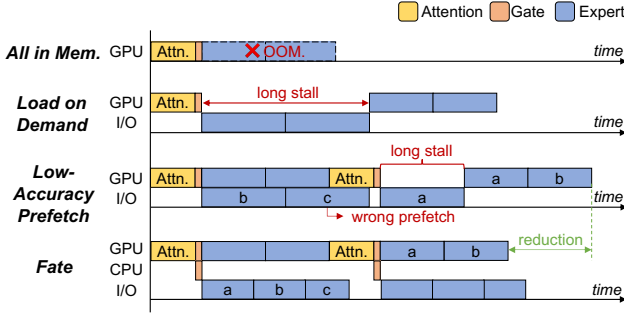


Figure 1: Fate and three kinds of pipeline.

with low resource requirements by activating only a small subset of parameters. For example, Qwen1.5-MoE [38] activates only 2.7 billion parameters during inference, providing better performance and faster inference speeds than many dense 7-billion-parameter models. However, MoE models are rarely used on edge due to their enormous parameter sizes. Their multi-expert architecture substantially increases memory requirements, making it infeasible to load all parameters simultaneously.

Offloading is an effective way to mitigate memory bottlenecks. It releases experts that are not activated from memory, significantly reducing memory requirements. However, loading experts on demand severely impacts inference latency, which is unacceptable in latency-sensitive scenarios, like shown in Figure 1. This is because expert activation depends heavily on the gating results, making it difficult to determine which experts to load in advance.

Existing studies propose various prefetching strategies, which can be broadly categorized into two approaches: expert activation path based methods and small network based methods. The path based approaches, such as Lina [24] and MoE-Infinity [41], leverage the statistical correlation (frequency relationship) between the experts selected by the tokens in consecutive layers. However, these methods suffer from low prediction accuracy, which leads to on-demand expert loading, as illustrated in Figure 1. On the other hand, small network based methods train lightweight networks to predict expert selection or replace original gates, achieving significantly higher accuracy [12, 18]. Nevertheless, these methods introduce additional training costs, lack portability, and often require fine-tuning to maintain performance. To enable fast inference of MoE models in edge scenarios, the following four challenges must be addressed.

C1: High-accuracy and low-cost expert prefetch. Accurate expert prefetching is the most critical challenge. Given the limited memory, offloading experts necessitates parallel prefetching to reduce latency. However, due to the dynamic gating mechanism, expert activation is determined by input features at runtime, making it unpredictable by nature. Existing expert prefetching methods [12, 41] either suffer from low prediction accuracy or incur high training costs.

C2: Expert caching with high hit rate. Expert caching is effective in further improving hit rates, but designing a high-hit-rate caching strategy is challenged by dynamic nature of gating. This making it difficult to determine which experts should be cached

within limited cache space. Existing methods [35, 41], such as LRU or static popularity-based caching, often underperform. This is because static strategies conflict with MoE’s dynamic routing and rarely account for layer-wise variations in caching needs.

C3: Overlap of computation and I/O in the pipeline. The I/O overhead during inference can have a significant impact on latency, yet the ability to efficiently overlap with computation is limited by bandwidth. For example, on an NVIDIA RTX 3090, Qwen1.5-MoE requires approximately 6 ms of I/O per expert, compared to around 13 ms of computation per decoder layer. This only allows for an effective overlap of up to two experts. Quantization is a common mitigation strategy. EdgeMoE [44] assigns bit widths per expert according to sensitivity. However, this static assignment method is not well suited to the dynamic routing inherent in MoE models.

C4: Differences between the prefill and decoding stages. The prefill and decoding stages differ in computation, expert activation, and resource demands, making unified optimization suboptimal. During prefill, many tokens are processed simultaneously, activating many experts at once, where minor prediction errors are tolerable, so the focus is on overlapping computation and I/O. In contrast, decoding is autoregressive, prioritizing accurate expert prediction over overlap. However, most existing works apply uniform strategies across both stages, overlooking these phase-specific requirements.

To address these challenges, we propose Fate, an efficient MoE inference system designed for edge scenarios, thereby facilitating the deployment of MoE models on resource-constrained environments. The key observation underlying Fate is that the input from the previous gate can be effectively utilized by the next gate for expert selection, enabling highly accurate expert prefetching. In addition, Fate identifies the lower prefetch accuracy in shallow layers and introduces a shallow-favoring expert caching strategy to improve expert cache hit rates. Finally, Fate leverages quantization to increase the number of cached experts and accelerate I/O operations, further improving overall inference efficiency.

In summary, this paper makes the following contributions:

- We propose a cross-layer expert prefetching strategy that achieves 97.15% accuracy. In addition, we introduce a shallow-favoring expert cache, which raises the cache hit rate to 99.08%. To further reduce I/O overhead while preserving inference accuracy, we present a popularity-aware hybrid quantization method.
- We implement the above strategies in Fate, an MoE-oriented inference system, which enables fast inference of MoE models in edge scenarios.
- To evaluate the performance of Fate, we conducted experiments to measure its impact on inference efficiency. The results show that, compared to the baselines, Fate achieves $1.34\times$ – $5.07\times$ prefill speedup and $1.26\times$ – $4.41\times$ decoding speedup, while maintaining high inference quality.

2 Background & Related Work

A Primer on MoE Model. To further scale the model size without increasing the training costs, some works introduced the sparsely activated MoE architecture [15, 33]. Due to its ability to maintain high computational efficiency at relatively low cost, MoE has rapidly

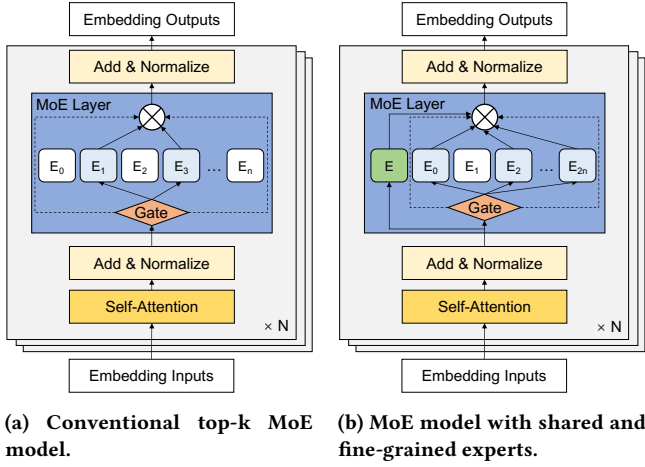


Figure 2: Two mainstream structural designs of MoE model.

become one of the mainstream architectures for LLMs. Representative examples include GPT-4 [1], Gemini 1.5 [36], and DeepSeek-V2 [11], etc. Currently, MoE models can be categorized into two main structural designs, as illustrated in Figure 2.

In the conventional top-k architecture, shown in Figure 2a, the FFN layer in the Transformer is replaced by an MoE layer consisting of a gating network and multiple experts. The gating mechanism is the core concept of MoE, which is typically implemented as a softmax function. The top-k experts are activated based on the gating results.

To further enhance expert specialization, DeepSeekMoE [9] introduces a shared and fine-grained expert architecture, as illustrated in Figure 2b. Each expert is divided into fine-grained experts, and more experts are activated to keep the computational cost stable. This subdivision improves the specialization level of each expert, allowing for more precise knowledge distribution and flexible combinations of experts. Additionally, a subset of experts is isolated as shared experts, which are always activated to capture general knowledge. In addition to DeepSeekMoE, other models such as DBRX [37], Qwen1.5-MoE [38], and OLMoE [26] also demonstrated the effectiveness of using fine-grained experts.

Offloading Systems. Offloading systems have been instrumental in enabling efficient LLM inference under resource constraints. For instance, DeepSpeed-Inference [2] pioneered the application of ZeRO series [30, 31] offloading techniques to LLM inference, facilitating resource-constrained deployments. FlexGen [34] uses linear programming to optimise computational graphs, significantly improving inference throughput. However, these offloading systems are primarily designed for dense models and do not take into account the sparse activation characteristics of MoE models. To address this gap, Mixtral-Offloading [14] is the first to enable inference of Mixtral-8×7B [19] on consumer-grade hardware. Similarly, MoE-Infinity [41] reduced the latency costs associated with expert offloading through activation-aware expert prefetching and caching strategies. Pre-gated MoE [18] trained a pre-gate to replace the original gating mechanism, enabling early routing decisions. However, these methods often suffer from trade-offs such as limited routing

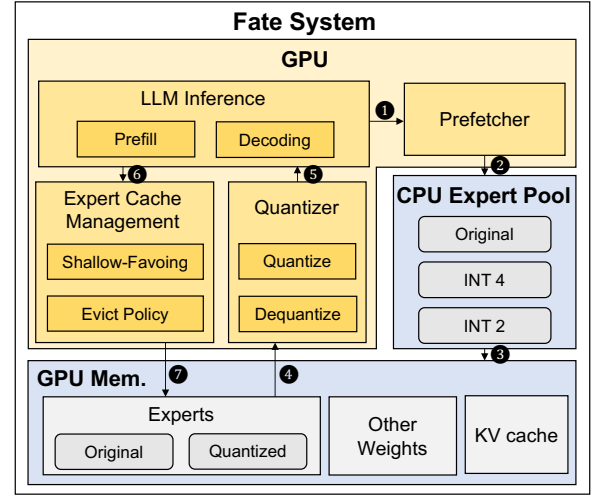


Figure 3: Overview of Fate.

accuracy or additional training overhead, which can reduce their practicality. In contrast, Fate overcomes the challenges of expert prediction with a simple yet highly accurate strategy, achieving efficient MoE inference under offloading.

3 Overview

To tackle the aforementioned challenges, we propose Fate, which is designed to facilitate rapid inference of MoE models at the edge. We show the system overview of Fate in Figure 3.

During the inference of MoE models, Fate adopts different strategies for the prefill and decoding phases to cope with their different resource requirements (1). Before each MoE block's gate computation, the current input is fed into the next layer's gate to predict required experts (2). This operation generates a prefetch list containing the indexes of the experts likely to be involved in the computation at the next layer, together with the quantization bit width for each expert. For each expert in the prefetch list, the CPU checks whether the expert is already exists in GPU memory. If an expert is not found in GPU memory, it is transferred from the CPU expert pool to GPU memory with the corresponding quantization bit width (3, 4). The GPU memory stores the dense weights, the KV cache and the cached experts. When the computation reaches the next layer on the GPU, the required experts are retrieved from the GPU memory (5), and the quantized experts are dequantized before being used in the computations (6). After completing the computations for the MoE layer, Fate updates the cache list based on the results of the gate (7). The Fate evicts experts that are not in the cache list to optimize memory usage (8).

4 Fate Design

4.1 Prefetching Opportunities

Start Point: Cosine Similarity. Fate's prefetching strategy is mainly based on the key observation that the inputs of neighbouring gates are similar. Specifically, inspired by InfiniGen [23], we calculate the cosine similarity between $Gate_in_{i+1}$ and the following

Table 1: Average cosine similarity and prefetch accuracy between $Gate_{in_{i+1}}$ and the following inputs: $Attn_{in_{i+1}}$, $Gate_{in_i}$ and $Attn_{in_i}$.

Position	Average Similarity	Prefetch Accuracy	
		Prefill	Decoding
1	93.05%		83.99%
2	88.83%	>99.6%	78.79%
3	83.84%		76.94%

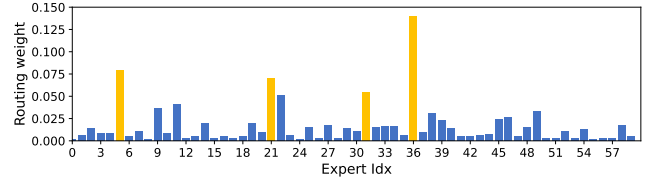
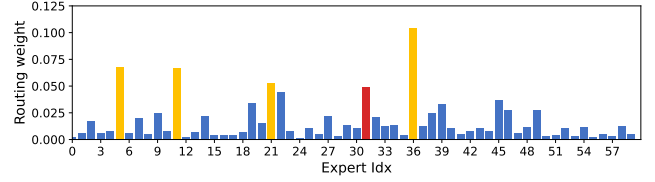
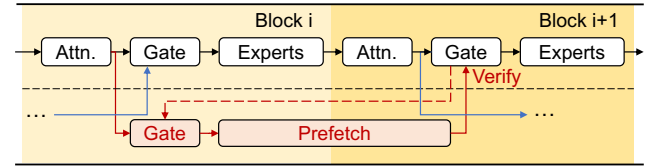
inputs: $Attn_{in_{i+1}}$ (position 1), $Gate_{in_i}$ (position 2) and $Attn_{in_i}$ (position 3). The results, as presented in Table 1, indicate that the average cosine similarity exceeds 83%, with a clear trend showing that the closer the position is to $Gate_{in_{i+1}}$, the higher the cosine similarity.

Then, we further try to use $Attn_{in_{i+1}}$, $Gate_{in_i}$, and $Attn_{in_i}$ for expert prediction. The average prediction accuracy is also shown in Table 1. During the prefill stage, numerous tokens are processed simultaneously, and each token selects its top-k experts. The overlap among these prediction results lead to an accuracy close to 100%. In contrast, during the decoding stage, only a single token is processed at each step, making the high prediction accuracy more meaningful. The results show that the lowest prediction accuracy reaches an impressive 76.94%. This result potentially surpasses the accuracy of some activation-aware prediction methods.

Among the three positions of the inputs, considering the trade-off between I/O time and prefetch accuracy, we choose $Gate_{in_i}$ at position 2. Although $Attn_{in_{i+1}}$ achieves the highest prefetch accuracy, the computation time of a single attention layer is short, making it difficult to complete the I/O. On the other hand, selecting $Attn_{in_i}$ offers a longer I/O time, but $Attn_{in_i}$ and $Gate_{in_{i+1}}$ are separated by more than one MoE block. This separation introduces a risk of I/O conflicts between adjacent layers. In addition, $Attn_{in_i}$ has the lowest prediction accuracy. Therefore, we select $Gate_{in_i}$, achieving a prefetch accuracy of 78.79% without additional training or statistical costs.

Similar Routing Weights. Although the above method is simple and effective, it still results in a misprefetch rate of over 20%. We know that I/O is one of the main causes of high latency in inference. Each misprefetch requires an on-demand transfer of experts, which is unacceptable in latency-sensitive scenarios. Therefore, to further improve prefetch accuracy, we delve deeper into the routing weights obtained from the gate computation.

The routing weights computed by the gate include the probability that each expert is selected. The top-k experts are selected based on the highest routing weights as the final output. We randomly select results of once misprefetch, and compare the distribution of routing weights obtained from the predictions with those obtained from the actual computations. The true results are shown in Figure 4a and predicted results are shown in Figure 4b. It is evident that the routing weight distributions in both figures are similar. And the missing expert 31 ranked fifth in the prediction results. This observation suggests that under similar routing weight distributions, increasing the number of prefetched experts could likely include the truly required experts. Based on this insight, we attempted to

**(a) The true routing weights using $Gate_{in_{i+1}}$.****(b) The predicted routing weights using $Gate_{in_i}$.****Figure 4: Comparison between true and predicted routing weights at block $i + 1$.****Figure 5: Workflow of cross-layer expert prefetch.**

transfer experts whose confidence scores exceed the 75th percentile, achieving an impressive prefetch accuracy of 97.15%.

4.2 Cross-Layer Expert Prefetch

Building on the prefetching opportunities observed in Section 4.1, Fate's expert prefetching workflow is illustrated in Figure 5. Specifically, Fate feeds the input into $Gate_{i+1}$ immediately before $Gate_i$ to trigger prefetching. As demonstrated in Section 5.5, the gate's minimal computational cost allows for direct execution on the GPU with negligible overhead. In contrast, offloading inputs to the CPU would introduce unnecessary latency. This strategy is remarkably elegant due to its simplicity, effectiveness, and broad applicability across different scenarios.

Prefill Stage. During the prefill stage, where all tokens in the sequence are processed simultaneously, a large number of experts are activated. Therefore, Fate focuses on minimizing pipeline bubbles between expert computations. Naive approach sequentially transfers experts and computes based on the order of token requests, which can lead to significant idle time. To address this, Fate statistically aggregates token-to-expert assignments after obtaining expert prediction results, thereby calculating the popularity of each expert. Experts are then transferred in descending order of popularity, prioritizing those involved in computations for a larger number of tokens. This approach ensures that the most frequently

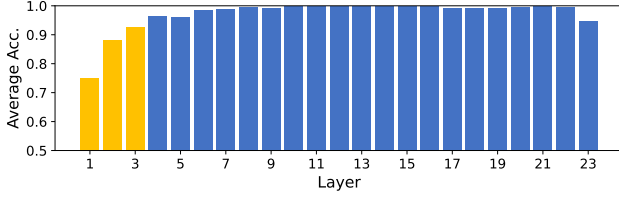


Figure 6: Average accuracy of expert prefetching at each layer. Model: Qwen1.5-MoE. $L = 3$

used experts are available earlier, providing additional time for subsequent expert I/O, thus improving pipeline efficiency.

Decoding Stage. During the decoding stage, where only a single token is processed at a time, the activated experts are limited to the top- k . At this stage, Fate prioritizes achieving full overlap between computation and I/O. First, since the available prefetching window corresponds to the computation time of one decoder layer, Fate statistically pre-computes the average decoder layer computation time during decoding to obtain the time constraint T . In addition, Fate also measures the I/O time for a single expert (t_{expert}). Based on the time constraint T , the maximum number of experts that can be transferred is calculated as follows:

$$n = \frac{T}{t_{expert}}$$

Importantly, this calculation of n is performed offline, ensuring that no additional computational overhead is introduced during the inference process. By pre-computing this, Fate maximizes the overlap between computation and I/O during decoding, effectively minimizing latency.

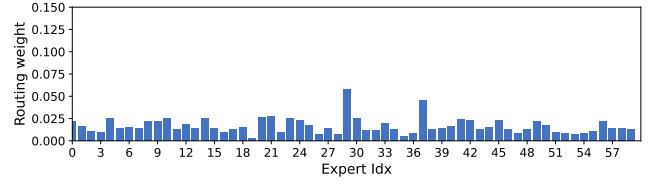
4.3 Shallow-Favoring Expert Cache

To further improve expert hit rates, Fate leverages available memory resources to cache experts, thereby reducing the I/O overhead. First, Fate defines a memory budget to constrain peak memory usage during inference, in order to handle resource contention among multiple applications in real-world scenarios. Fate then models the memory usage based on the memory footprint of each layer, KV cache and activation, etc. After subtracting the memory occupied by dense parts, the remaining memory is allocated for expert caching. The total number of experts that can be cached ($cache_total$) is derived using the following formula:

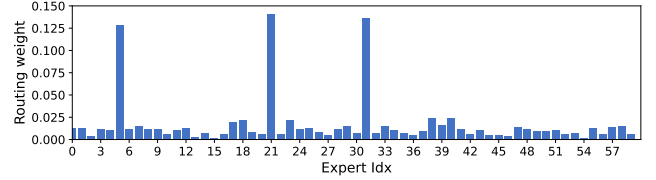
$$cache_total = \frac{memory_budget - others}{expert}$$

where $memory_budget$ refers to the total memory allocated for inference. $others$ refers to memory consumed by non-expert parts (e.g., dense layers, KV cache, etc). $expert$ refers to the memory required per expert.

In order to allocate $cache_total$ more effectively across different layers, we calculate the average prefetch accuracy for each layer based on the findings in Section 4.1. As shown in Figure 6, the prefetch accuracy in shallow layers is obviously lower than in deeper layers. To understand this discrepancy, we further examined the differences in routing weights between shallow and deep layers,



(a) Routing weights of the first layer.



(b) Routing weights of the 23rd layer.

Figure 7: Routing weights of the first layer and the 23rd layer.

as shown in Figure 7. Comparing Figure 7a and Figure 7b, shallow layers exhibit relatively uniform expert routing, whereas deeper layers reveal stronger preferences, with tokens more decisively assigned to specific experts. Consequently, inputs with high cosine similarity yield higher prediction accuracy in deeper layers. Motivated by this observation, we conducted a simple experiment: we cached all experts in layers 0-3. The results showed an average expert hit rate of 99.08%, demonstrating the effectiveness of targeted caching in shallow layers.

Based on the experimental results, Fate introduces the shallow-favoring expert cache strategy. Specifically, Fate sets a boundary L between shallow and deep layers based on offline statistical analysis, and prioritises the allocation of memory resources to shallow layer, while the remaining memory is distributed evenly across the deep layers. For example, from the observed data in Figure 6, the boundary for Qwen1.5-MoE is determined as $L = 3$. If memory resources are limited, Fate allocates expert cache space sequentially to the first L layers. If memory resources are sufficient, all shallow layer experts are fully cached, and the cache space for each deep layer is calculated as:

$$cache_deep = \frac{cache_total - L \times num_experts}{num_layers - L}$$

where $num_experts$ is the number of experts per layer, and num_layers is the total number of layers in the model.

To manage expert caching within each layer, Fate uses the Adaptive Replacement Cache (ARC) algorithm for expert eviction. While the commonly adopted approach is LRU caching, observations of the popular experts [24] indicate that activation frequency is also a critical factor in determining the importance of an expert during MoE inference. ARC dynamically balances both recency and frequency, ensuring efficient cache management under varying access patterns. Specifically, the ARC algorithm maintains four key queues: T1 (recently accessed cache experts), T2 (frequently accessed cache experts), B1 (experts evicted from T1), and B2 (experts evicted from T2). By dynamically adjusting the sizes of T1

Table 2: Impact of quantization on the inference accuracy of Qwen1.5-MoE. "Acc.": accuracy; "Degrad.": accuracy degradation compared to original accuracy. Benchmark: MMLU.

Model	Qwen-1.5-MoE			
	Original	INT8	INT4	INT2
Acc.	59.6	59.6	59.4	55.1
Degrad.	0	0	-0.2	-4.5

and T2, ARC can adaptively switch strategies in response to changing access patterns. For example, in short-term hotspot scenarios, ARC expands T1 to prioritize recently accessed experts, ensuring immediate availability. Conversely, in long-term or frequent access scenarios, ARC prioritizes T2, preserving high-frequency experts for sustained reuse.

4.4 Expert Quantization Management

To accelerate I/O operations and reduce the memory footprint of expert caching, Fate applies quantization to experts. After evaluating a range of popular quantization algorithms, we selected HQQ (Half-Quadratic Quantization [4]), which has been demonstrated to achieve excellent performance on MoE models while offering fast quantization and dequantization speeds.

Quantization for Cache. During expert caching, Fate employs int4 quantization when memory resources are insufficient. Existing research indicates that MoE experts exhibit strong robustness to quantization [21]. As shown in Table 2, even with INT4 quantization, the accuracy degradation remained remarkably low. Considering the trade-off between memory consumption and accuracy loss, we opted for INT4 quantization. This choice significantly reduces the memory footprint per expert, allowing a substantial increase in the total number of cached experts (*cache_total*). When prefetched experts are cached in INT4 format, Fate performs dequantization, which is faster than directly loading the experts.

Quantization for I/O. During the expert transfer, Fate employs different quantization strategies tailored to the characteristics of the prefill and decoding stages. In the prefill stage, Fate introduces a popularity-aware hybrid quantization strategy. Since all tokens are processed simultaneously, a large number of experts are activated, which imposes significant I/O pressure. However, treating all experts equally during I/O may not be optimal. As shown in Figure 8, we ranked all experts in descending order of popularity and analyzed the distribution of tokens among the across non-popular experts. The results show a clear imbalance between the number of non-popular experts and the tokens they cover. For instance, in layer 6, the orange section indicates that 30% of the active experts cover only 15% of the tokens. Therefore, it may not be worth transferring these non-popular experts for the same I/O cost.

Furthermore, we quantize experts covering different proportions of tokens into INT2, as illustrated in Figure 9. It is evident that when experts are quantized in descending order of popularity, converting non-popular experts from INT4 to INT2 has minimal impact on inference quality. In contrast, quantizing popular experts to INT2 results in a significant drop in performance scores. Accordingly, Fate

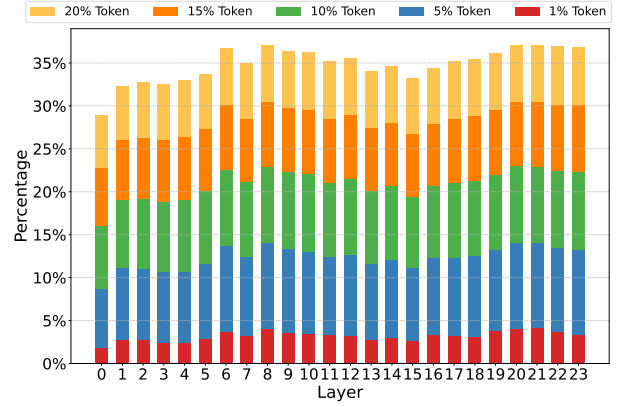


Figure 8: The distribution ratio of the tokens among the experts in each layer. The y-axis represents the percentage of experts that process this part of the tokens.

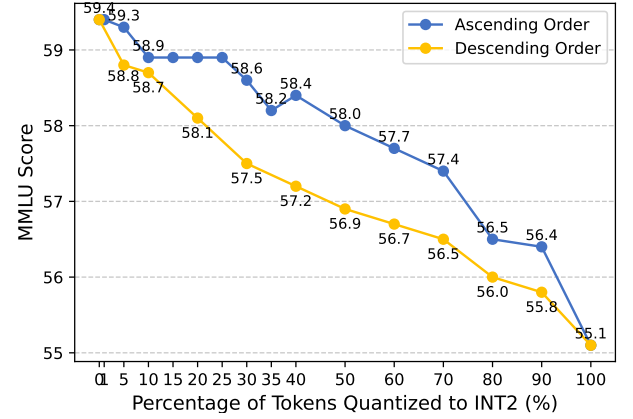


Figure 9: The trend of MMLU scores as the percentage of INT2 experts increases. The blue line shows the experts quantized in reverse order of popularity, and the yellow line shows the situation in positive order. Model: Qwen1.5-MoE.

stores both INT2 and INT4 format of all experts in CPU memory and dynamically transfers the appropriate format during prefill based on predicted popularity. Fate uses a heuristic search to determine the optimal INT2 proportion p for each model while keeping accuracy loss under 1% (e.g., $p = 25\%$ for Qwen1.5-MoE). In contrast, during decoding, where each expert is equally important, Fate transfers the INT4 format.

5 Evaluation

5.1 Experimental Setup

Hardware. We conducted our experiments on a PC equipped with an NVIDIA RTX 3090 GPU (24 GB). It's used to simulate common edge-side scenarios with either 24 GB (Env 1) or 16 GB (Env 2) of GPU memory. The CPU and GPU were interconnected via PCIe 3.0 $\times 16$ (16GB/s bandwidth).

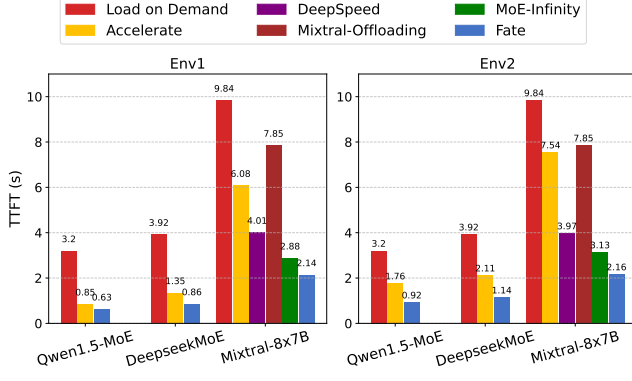


Figure 10: TTFT of Fate and baselines.

Models. We evaluate Fate on three popular MoE models: Qwen1.5-MoE (14.3B), DeepSeekMoE (16.4B), and Mixtral-8×7B (46.7B). The first two employ state-of-the-art shared-expert and fine-grained expert architectures, whereas Mixtral-8×7B adopts the conventional top-2 routing mechanism.

Workload. For end-to-end performance evaluation, we sample inputs from the ShareGPT [10] dataset. ShareGPT is constituted of authentic user interactions with ChatGPT and functions as a representative dataset for real-world LLM services. For inference accuracy evaluation, we use three widely recognized benchmarks: MMLU [17], GSM8K [8], and HumanEval [7], to analyze the impact of Fate’s quantization strategy on accuracy.

Baseline. Since most existing work primarily supports Mixtral-8×7B, we conduct a more comprehensive comparison on Mixtral-8×7B with multiple baselines, including Load on Demand, Hugging Face Accelerate [16], DeepSpeed-MII [25], Mixtral-Offloading [14], and MoE-Infinity [41]. At the same time, we compare our approach against Load on Demand and Accelerate on Qwen1.5-MoE and DeepSeekMoE.

Key Metrics. Our primary focus is on achieving fast inference of MoE models. Therefore, we evaluate end-to-end performance using two key metrics: time to first token (TTFT) and time per output token (TPOT). In addition, we use inference accuracy to evaluate the impact of quantization.

5.2 End-to-End Performance

5.2.1 Prefill Performance. We first evaluate Fate’s prefill performance, as shown in Figure 10. Fate achieves the lowest TTFT and a $1.34\times$ – $5.07\times$ speedup over the baselines, demonstrating substantial performance advantages. The acceleration achieved by Fate is primarily due to its strategies tailored to the prefill stage. Unlike other methods, Fate identifies that the prefill stage of MoE offloading inference is I/O intensive. Therefore, Fate reorders execution based on expert popularity, leveraging the longer computation times of popular experts to overlap the I/O of other experts, thereby reducing the idle time between experts. This reordering process is supported by highly accurate expert prefetching, which ensures a reliable popularity-based sequence. Moreover, Fate introduces

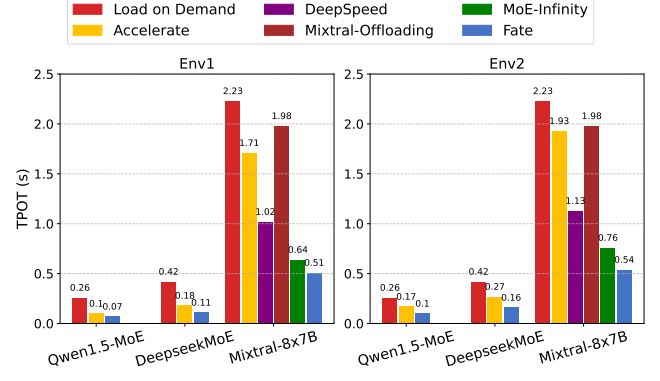


Figure 11: TPOT of Fate and baselines.

a popularity-aware hybrid quantization strategy, quantizing non-popular experts to INT2, to further alleviate I/O pressure while maintaining inference quality.

5.2.2 Decoding Performance. Figure 11 illustrates the comparison of TPOT. Across all settings, Fate achieves a $1.26\times$ – $4.41\times$ speedup in decoding latency over the baselines, demonstrates a clear performance advantage over the other strategies. During the decoding stage, each forward processes only one single token, making decoding performance highly dependent on the effectiveness of expert prefetching and expert caching. Fate significantly improves expert hit rates by leveraging high-accuracy cross-layer expert prefetching and shallow-favoring expert caching strategies. These mechanisms effectively overlap substantial I/O overhead, reducing latency. In contrast, baselines exhibit lower prefetching accuracy, making them prone to temporary I/O bottlenecks that lead to higher inference latency and substantially reduced decoding speed compared to Fate. Consequently, Fate enables fast inference of MoE models in the offloading scenario.

5.3 Inference Accuracy

We evaluate the impact of Fate’s quantization strategy on inference accuracy across three representative benchmarks. Since Fate adaptively determines both the number of cached experts and their bit-widths according to the hardware environment, we conduct evaluations separately in Env 1 and Env 2. The results obtained on each benchmark are compared with the original model’s accuracy scores, and the average loss is calculated. As shown in Table 3, Fate achieves acceptable reductions in inference quality while maintaining high efficiency.

Specifically, Fate achieves an average accuracy loss of less than 3% compared to the original FP16 models, with slight improvements observed on certain benchmarks. This is due to Fate’s ability to accurately identify experts that handle fewer tokens and quantize them to INT2, which not only improves I/O efficiency but also has minimal impact on inference quality. Evidently, in Env 2, the limited 16 GB of GPU memory forces more experts to be quantized into int2 for fast temporary I/O in cases of mis-prefetching, which in turn leads to higher loss. However, this marginal accuracy trade-off is justified by a decoding speedup of over $1.26\times$. Overall, these results

Table 3: The impact of Fate on the inference accuracy of three models on three benchmarks.

MMLU (5-shot)	BF16	Env 1	Env 2
Qwen1.5-MoE	59.6	59.7	59.9
DeepseekMoE	44.4	44.2	44.2
Mixtral-8×7B	70.5	69.3	68.9
Avg. Loss	/	-0.66%	-0.74%
GSM8K (5-shot)	BF16	Env 1	Env 2
Qwen1.5-MoE	61.5	60.2	59.9
DeepseekMoE	21.2	21.4	21.5
Mixtral-8×7B	64.9	63.2	60.3
Avg. Loss	/	-1.26%	-2.76%
HumanEval (0-shot)	BF16	Env 1	Env 2
Qwen1.5-MoE	34.2	33.3	33.5
DeepseekMoE	25.6	27.4	25.0
Mixtral-8×7B	32.3	34.1	36.0
Avg. Loss	/	+3.32%	+2.35%

demonstrate that Fate can effectively balance inference performance with accuracy.

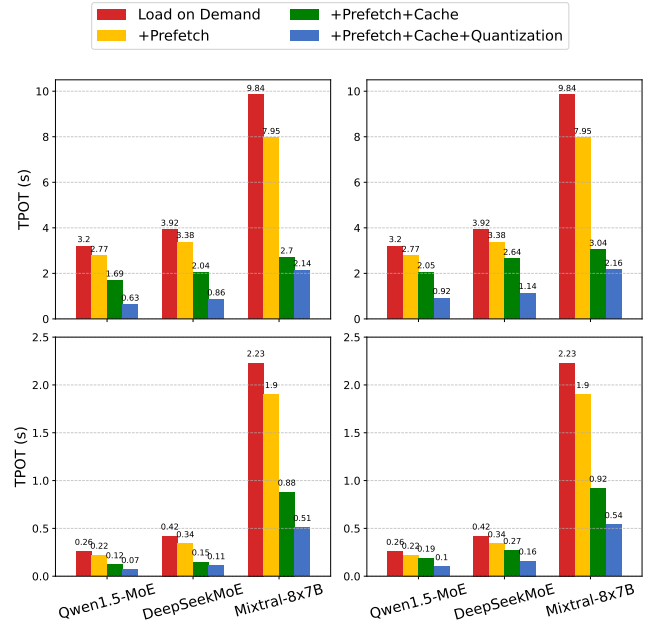
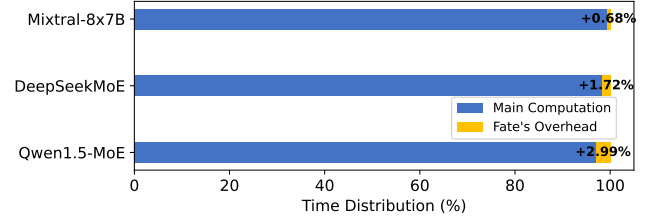
5.4 Ablation Study

We conducted a comprehensive breakdown analysis of the benefits brought by each technique in Fate as shown in Figure 12. Starting with Load on Demand as a naive baseline, we incrementally incorporated the prefetching strategy (§ 4.2), the caching strategy (§ 4.3), and the quantization strategy (§ 4.4). We then compared their respective contributions to reducing TTFT and TPOT.

Specifically, in each scenario, the TTFT and TPOT decreases progressively with each added component, directly demonstrating the effectiveness of each strategy. First, prefetching achieves an average 16% speedup in both TTFT and TPOT, confirming the high effectiveness of the prefetching strategy. The caching strategy, by caching a significant number of experts, effectively reduces I/O overhead. Building on prefetching, caching further reduces latency by 35% in TTFT and 33% in TPOT. Quantization, in addition to reducing I/O overhead, allows more experts to be cached and achieves the most significant improvement, delivering up to a 4.3 prefill speedup and 3.5× decoding speedup compared to Load on Demand. Overall, these results highlight the complementary benefits of prefetching, caching, and quantization in improving inference efficiency.

5.5 Overhead

Fate introduces two potential execution stalls: gating computation and cache queue updates. To quantify this, we measure its runtime overhead across three MoE models. As shown in Figure 13, the yellow segment denotes the overhead fraction, which remains consistently negligible compared to the full inference forward. Specifically, the measured overhead accounts for only 0.68% in Mixtral-8×7B, 1.72% in DeepSeekMoE, and 2.99% in Qwen1.5-MoE. These results

**Figure 12: Ablation study of Fate.****Figure 13: Overhead analysis of Fate.**

demonstrate that Fate introduces only a marginal time cost, ensuring that the efficiency gains from Fate are not offset by overhead.

6 Conclusion

This paper presents Fate, a system designed for fast MoE inference in edge scenarios. The key insight of Fate lies in leveraging the high cosine similarity of adjacent gate inputs to achieve accurate cross-layer prefetching. In addition, a shallow-favoring expert caching strategy is designed to improve the expert hit rate to over 99%. Quantization techniques are also integrated to optimize both I/O and caching. Experimental results show that Fate effectively accelerates the prefill and decoding speed of MoE models while maintaining inference accuracy.

Acknowledgments

This work was supported in part by the Major Key Project of PCL under Grant PCL2025A03, the National Natural Science Foundation of China (62472459), CAAI-MindSpore Open Fund, developed on OpenI Community(CAAIXSJLJ 2023 MindSpore 01).

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. 2022. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [3] Anthropic. 2024. Introducing the next generation of Claude. <https://www.anthropic.com/news/claude-3-family> <https://www.anthropic.com/news/claude-3-family>
- [4] Hicham Badri and Appu Shaji. 2023. Half-Quadratic Quantization of Large Machine Learning Models. https://mobiusml.github.io/hqq_blog/ https://mobiusml.github.io/hqq_blog/
- [5] Hongru Cai, Yongqi Li, Wenjie Wang, Fengbin Zhu, Xiaoyu Shen, Wenjie Li, and Tat-Seng Chua. 2025. Large language models empowered personalized web agents. In *Proceedings of the ACM on Web Conference 2025*, 198–215.
- [6] Lin Chen, Fengli Xu, Nian Li, Zhenyu Han, Meng Wang, Yong Li, and Pan Hui. 2024. Large language model-driven meta-structure discovery in heterogeneous information network. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 307–318.
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgren Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374 [cs.LG]*
- [8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021).
- [9] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y Wu, et al. 2024. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066* (2024).
- [10] Hugging Face Dataset. 2023. ShareGPT. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered
- [11] DeepSeek-AI. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv:2405.04434 [cs.CL]*
- [12] Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai Li, and Yiran Chen. 2024. SiDA: Sparsity-Inspired Data-Aware Serving for Efficient and Scalable Large Mixture-of-Experts Models. *Proceedings of Machine Learning and Systems* 6 (2024), 224–238.
- [13] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [14] Artyom Eliseev and Denis Mazur. 2023. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238* (2023).
- [15] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [16] Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. 2022. Accelerate: Training and inference at scale made simple, efficient and adaptable. <https://github.com/huggingface/accelerate>.
- [17] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. *Proceedings of the International Conference on Learning Representations (ICLR)* (2021).
- [18] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1018–1031.
- [19] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
- [20] Veton Kepuska and Gamal Bohouta. 2018. Next-generation of virtual personal assistants (microsoft cortana, apple siri, amazon alexa and google home). In *2018 IEEE 8th annual computing and communication workshop and conference (CCWC)*. IEEE, 99–103.
- [21] Young Jin Kim, Raffi Fahim, and Hany Hassan Awadalla. 2023. Mixture of Quantized Experts (MoQE): Complementary Effect of Low-bit Quantization and Robustness. *arXiv preprint arXiv:2310.02410* (2023).
- [22] Hanyu Lai, Xiao Liu, Iat Long Long, Shuntian Yao, Yuxuan Chen, Pengbo Shen, Hao Yu, Hanchen Zhang, Xiaohan Zhang, Yuxiao Dong, et al. 2024. Autowebglm: A large language model-based web navigating agent. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5295–5306.
- [23] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 155–172.
- [24] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating distributed MoE training and inference with lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 945–959.
- [25] Microsoft. [n. d.]. Deepspeed-MIL. <https://github.com/microsoft/DeepSpeed-MIL>
- [26] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, et al. 2024. OLMoE: Open Mixture-of-Experts Language Models. *arXiv preprint arXiv:2409.02060* (2024).
- [27] Guangtao Nie, Rong Zhi, Xiaofan Yan, Yufan Du, Xiangyang Zhang, Jianwei Chen, Mi Zhou, Hongshen Chen, Tianhao Li, Ziguang Cheng, et al. 2024. A hybrid multi-agent conversational recommender system with LLM and search engine in e-commerce. In *Proceedings of the 18th ACM Conference on Recommender Systems*, 745–747.
- [28] Qualcomm. 2023. World's first on-device demonstration of Stable Diffusion on an Android phone. <https://www.qualcomm.com/news/onq/2023/02/worlds-first-on-device-demonstration-of-stable-diffusion-on-android> <https://www.qualcomm.com/news/onq/2023/02/worlds-first-on-device-demonstration-of-stable-diffusion-on-android>
- [29] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*. PMLR, 18332–18346.
- [30] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*, 1–14.
- [31] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. Zero-offload: Democratizing {billion-scale} model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, 551–564.
- [32] Chenyang Shao, Xinyuan Hu, Yutang Lin, and Fengli Xu. 2025. Division-of-thoughts: Harnessing hybrid language model synergy for efficient on-device agents. In *Proceedings of the ACM on Web Conference 2025*, 1822–1833.
- [33] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538* (2017).
- [34] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.
- [35] Xiaoni Song, Zihang Zhong, Rong Chen, and Haibo Chen. 2024. Promoe: Fast moe-based llm serving using proactive caching. *arXiv preprint arXiv:2410.22134* (2024).
- [36] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530* (2024).
- [37] Mosaic AI Research Team et al. 2024. Introducing DBRX: a new state-of-the-art open LLM. Mosaic AI Res. Available online at: <https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm> (accessed June 4, 2024) (2024).
- [38] Qwen Team. 2024. Qwen1.5-MoE: Matching 7B Model Performance with 1/3 Activated Parameters. <https://qwenlm.github.io/blog/qwen-moe/> <https://qwenlm.github.io/blog/qwen-moe/>
- [39] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. 2024. Empowering 1000 tokens/second on-device llm prefilling with mllm-npu. *arXiv preprint arXiv:2407.05858* (2024).
- [40] Haoran Xu, Young Jin Kim, Amr Sharaf, and Hany Hassan Awadalla. 2024. A Paradigm Shift in Machine Translation: Boosting Translation Performance of Large Language Models. In *The Twelfth International Conference on Learning Representations*.

- [41] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. Moe-infinity: Activation-aware expert offloading for efficient moe serving. [arXiv preprint arXiv:2401.14361](#) (2024).
- [42] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024. Qwen2 technical report. [arXiv preprint arXiv:2407.10671](#) (2024).
- [43] Yuan Yao, Tianyu Yu, Ao Zhang, Chongyi Wang, Junbo Cui, Hongji Zhu, Tianchi Cai, Haoyu Li, Weilin Zhao, Zhihui He, et al. 2024. Minicpm-v: A gpt-4v level mllm on your phone. [arXiv preprint arXiv:2408.01800](#) (2024).
- [44] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2023. Edgemoe: Fast on-device inference of moe-based large language models. [arXiv preprint arXiv:2308.14352](#) (2023).